

ENTERPRISE AGENTIC AI · MODULE 01

企业级 Agentic AI 落地与 Harness 架构实战

模块一 · 企业级 Agentic 架构演进与 Workshop 底座剖析

从 RAG 单轮到自主 ReAct 闭环 · 基于 EnterpriseAgenticWorkshop · 时长 40 分钟

本模块要解决的问题

三个问题，定义本模块的边界

01

传统 AI 应用为何撑不起“自主完成任务”？

02

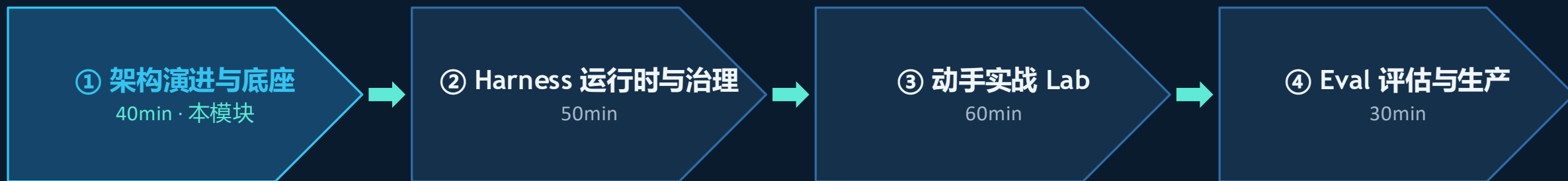
多个智能体之间到底如何分工与协作？

03

Workshop 这套企业级代码底座长什么样？

结论 本模块用 40 分钟把“架构演进 + 工程底座”一次讲透。

四站式路线图：本模块是第一站



全景 全系列共 180 分钟，围绕“多智能体编排 + Agent Harness 治理 + 评测上线”。

三个目标：看懂演进、拓扑与底座

演进认知

讲清 RAG / 单轮 → ReAct 自主闭环的技术跨越，看懂能力边界的迁移。

拓扑范式

掌握 Supervisor 与 Peer-to-Peer 两种多智能体协作模式及取舍。

工程底座

读懂 Workshop 目录结构、6 大模块与 MCP 通信协议如何落地。

路径 先建立概念地图，再逐层落到可运行的工程实现。



PART 01 · 第一部分

从传统 AI 到自主 Agent —— 一次执行范式的跃迁

传统 AI 的三种范式仍是一问一答

单轮对话

一次用户输入 → 一次模型调用 → 一次回答；无状态、无工具。

RAG 检索增强

先检索知识、再生成，缓解幻觉，但仍是“一问一答”。

工具增强 Function Calling

能调外部函数，但缺乏多步规划与自我纠错闭环。

共性 三者都无法自主拆解目标、闭环使用工具并自我纠错。

单轮与 RAG 撑不起复杂任务闭环

三处结构性缺口

复杂目标需要拆解、排序、条件分支，而一次性生成无法完成这些动作。

- 缺乏多步规划：复杂目标需拆解、排序与条件分支。
- 无工具反馈闭环：拿到结果后不能重新决策、重试、修复。
- 无持久状态：长流程一断即失，无法断点续跑。

结论 企业级复杂任务需要“会规划、会用工具、会反思”的自主体。

Agentic AI 是会自主决策的执行体

自主性

面向目标自行决定下一步做什么。

工具使用

把 LLM 当“大脑”，通过工具与真实系统交互。

反思与修复

观察结果、发现错误、自我纠正后继续。

一句话 Agent = 目标 + 规划 + 工具 + 记忆 + 反思 的循环。

ReAct 用思考-行动-观测构成闭环

Reasoning 思考

基于目标与上下文推理下一步该做什么。

Acting 行动

选择并调用合适的工具或子任务。

Observing 观测

读取工具返回、更新认知，再进入下一轮。

循环直到目标达成或触发终止条件——这是自主执行的最小闭环。

每一轮观测都可能触发新一次推理



侧注

目标 → 思考 → 行动 → 观测 → 回到思考
..... 直至完成。

每一轮都可能触发一次或多次模型调用——为后续“Token 经济学”埋下伏笔。

规划、工具、记忆、反思四支柱缺一不可

规划 Planning

任务拆解、排序与条件分支。

工具 Tools

Function Calling / MCP 接入外部能力。

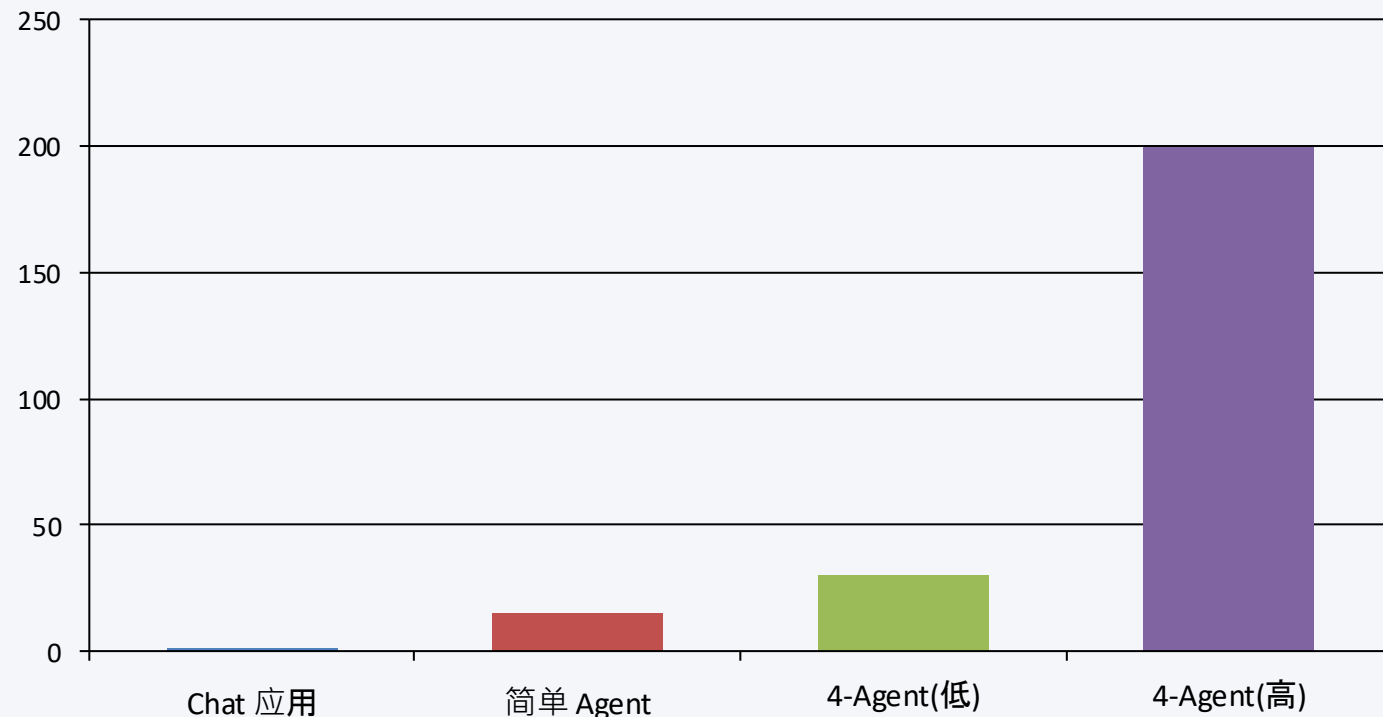
记忆 Memory

短期工作记忆 + 长期知识沉淀。

反思 Reflection

自检、纠错与自愈。

从 Chat 到 Agent，调用量级陡增



数量级 (示意)

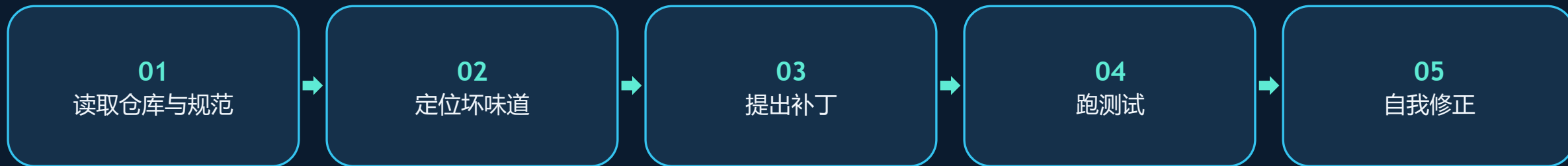
Chat: 1 次输入 $\approx$ 1 次调用。

Agentic: 规划→检索→选工具→执行→解读→反思→修复→总结，幕后可达数十次调用。

典型区间：30–200 次/任务。

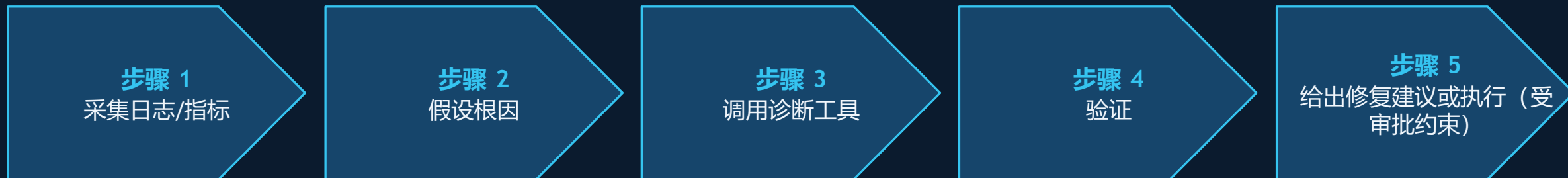
这既是能力也是成本——模块四将展开 Token 经济学。

代码审查从人工返工变自动闭环



场景一 · 价值 把“评审 + 返工”从人工循环变为可治理的自动循环。

DevOps 诊断把 SRE 经验沉淀为技能



场景二 · 价值 缩短 MTTR，把 SRE 的排障经验沉淀为可复用技能。

复杂分析从一问一答升级为多步推理

做法

多步检索 + 交叉验证 + 结构化归纳，回答“需要多份文档、多轮推理”的复杂问题。

价值

从“一问一答”升级为“可追溯的多步分析”，让结论有据可查。

场景三 知识库分析师：把零散检索整合为一条可复现的推理链。

多智能体用角色拆分换取可治理性

维度	单智能体	多智能体系统 (MAS)
职责	一体承担全部	按专长拆分角色
可控性	简单但易失控	角色 + 白名单更可治理
扩展性	受单体上下文限制	可并行 / 分工 / 最小权限
适用	简单任务	复杂业务流水线



PART 02 · 第二部分

多智能体协作拓扑：Supervisor 与 Peer-to-Peer

Supervisor 以中心协调换取可审计

- 一个 Supervisor / Planner 拆解目标。
- 把子任务分发给专门 Agent，再汇总结果。
- 编排清晰、易审计、易加护栏。

注意

Supervisor 既是控制点也是瓶颈——需防单点故障与上下文膨胀。

取舍 适合强流程、强治理场景：所有决策都经过中心可被审计。

中心协调者拆解并双向分发子任务



箭头双向：向下分发子任务，向上回报结果与状态。

Peer-to-Peer 以消息总线自适应交接

- 智能体间通过事件 / 消息总线交接（Handoff）。
- 无强中心，交接自适应触发。
- 灵活、可扩展、去中心。

注意

需要明确的交接契约与可观测性，否则链路难以追踪与调试。

取舍 适合高动态、松耦合场景：以契约与可观测性换取灵活度。

对等节点在总线上完成 Handoff



两种拓扑在控制与弹性间各有取舍

维度	Supervisor	Peer-to-Peer
控制	中心协调	去中心自适应
审计	天然集中易审计	需额外追踪
弹性	中心为瓶颈	无单点但更难调试
适用	强流程 / 强治理	高动态 / 松耦合

交接结构化契约而非整段对话历史

反模式

把整段对话历史塞给下一个 Agent → 上下文膨胀、成本飙升。

正确做法

需求 Agent 交出结构化 spec；编码 Agent 只读 spec；测试 Agent 只消费可测产物。

结论 交接的是“契约 / 产物”，不是“聊天记录”。（模块二 / 四深入）



PART 03 · 第三部分

EnterpriseAgenticWorkshop 底座剖析

Workshop 用可运行代码承载每个概念

定位

面向工程团队、构建生产级企业 Agent 的动手工作坊（约 6-8 小时 代码 + 幻灯片）。

路线

从单进程技能测试原型 → 多智能体沙箱流水线 → Foundry 托管 Agent（完整 Azure 基础设施）。

特点

代码优先：幻灯片里的每个概念都有可运行的对应代码。

主线 一条主线：把架构概念逐步落成生产级、可运行的工程实现。

四大工程关切点覆盖定义到部署

技能定义与对抗测试

Skill_Testing_Agent / AgentHarness_HostedAgent

沙箱化多智能体编排

OpenClaw_AgentHarness / CodingAgent

托管运行时与 Harness 架构

Brain / Hands / Session / Vault

云端部署到 Microsoft Foundry

skill-testing-harness-deploy

六个顶层目录对应六个可运行模块

Skill_Testing_Agent

对抗式技能测试

OpenClaw_AgentHarness

多智能体沙箱流水线

CodingAgent

上下文优化 + 纠错

AgentHarness_HostedAgent

Foundry 托管四层架构

skill-testing-harness-deploy

azd 上云部署

ppt

配套讲义幻灯片

每个目录对应一个可运行模块 + 配套幻灯片。

Skill Testing 用对抗测试打磨技能

- 最小化 Microsoft Agent Framework + GitHub Copilot SDK (Python)。
- 业务 Agent: 教育短视频脚本生成器, 带严格输出模板。
- 对抗测试 Agent: 构造边界用例挑战业务 Agent。

并行对比

两个模型 (Claude Opus 4.7 与 GPT-5.5) 并行运行, 在控制台直接对比表现。

定位 对抗式技能测试 101: 先把“技能是否可靠”问清楚。

先想清技能、评测与模型差异

业务 Agent

带确定性输出契约

对抗 Agent

构造单轮/多轮边界用例

确定性校验器

不用 LLM 即可校验契约

LLM-as-judge

按 5 个维度打分评审

边界用例集

10 个精选边界知识用例

先想清再落地

先定义评测，再谈基础设施

OpenClaw 以沙箱托起多智能体协作

- 一个 Docker-Compose 栈，引入 OpenClaw 作为沙箱 / 网关。
- 三个最小权限 Agent 在共享工作区内“自闭环”协作。
- 模型经 GitHub Copilot 内置 provider（Claude Opus 4.7）。

关键词

沙箱隔离 · 最小权限 · 共享工作区 · 网关凭据。

进阶 从单进程原型迈向真正的多智能体沙箱流水线。

只有 Runner 持有受约束的执行权

角色	职责	工具白名单
A — Coder	读 SPEC.md, 写 solution.py	read · write · edit
B — Tester	读 spec+solution, 写 test_solution.py	read · write · edit
C — Runner	沙箱内跑 pytest, 写 RUN_REPORT.md	read · write · exec

关键 只有 C 持有 exec, 且被 tools.exec.allowedPaths 约束到单一子目录。

把能力写死，越权在架构上不可能

教什么

工具白名单 · 工作区隔离 · 基于网关的凭据轮换 · 自纠错循环。

核心直觉

把“谁能做什么”写死成能力清单，越权在架构层面就不可能发生。

理念 安全不是运行时补丁，而是架构级的能力边界。

CodingAgent 聚焦上下文与自愈纠错

- 更进阶的 OpenClaw 流水线，聚焦两项生产技术。
- 三角色：coder · runner · diagnoser，各自严格白名单。
- 在控制 Token 花费的前提下扩展自愈式编码循环。

两项生产技术

- ① 上下文优化（按需加载）
- ② 错误纠正（diagnoser 驱动）。

目标 目标：更省 Token、更能自愈的可扩展编码循环。

按需加载技能文档，压缩上下文窗口

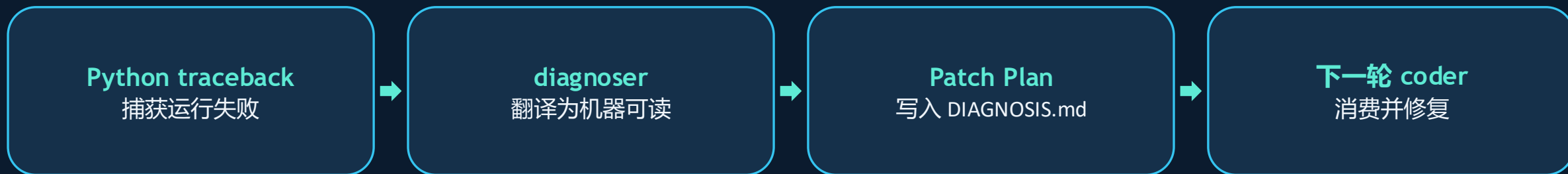
- coder 只加载 spec 用 @FILE.md 语法显式引用的技能文档。
- 如 PYTHON_STYLE.md、ALGO_PATTERNS.md、ERROR_HANDLING.md、TESTING.md。
- 让 prompt 窗口精简、可预测。

承前启后

这正是模块四“上下文压缩”的工程雏形——把窗口当成需要治理的预算。

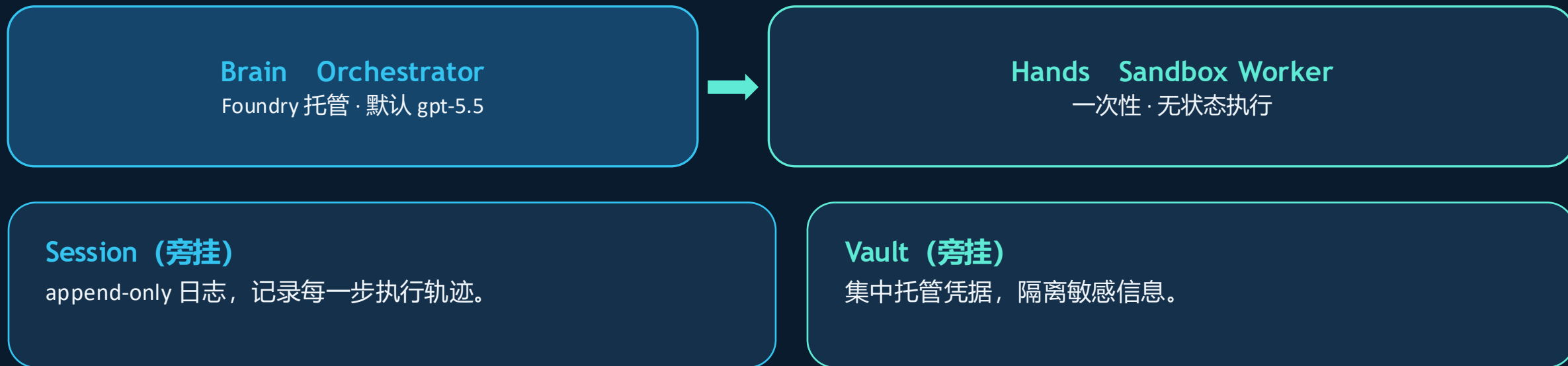
效果 显式引用 + 按需加载 = 更小、更稳定的上下文。

失败即修复：diagnoser 驱动纠错循环



闭环回流：一个“失败即修复”的可扩展自愈循环。

Foundry 托管四层：Brain 与 Hands 分离



旗舰模块：在托管 harness 上重写 Skill Testing 逻辑；细节留给模块二深入。

一条 azd up 把 Harness 部署上云

- azure.yaml + Bicep, 把 harness 作为 hosted agent 部署。
- host: azure.ai.agent, 用 azd up 一键上云。
- 展示完整 IaC 与远程构建流程。

IaC 覆盖

容器资源 · 启动命令 · 远程构建 · azure.ai.agents 扩展。

收益 从本地到云端：基础设施即代码，可复现、可审计。

一天议程：从概念到部署逐层递进



节奏 概念 → 沙箱 → 实践 → 托管 → 部署：一天走完从原型到上云的全链路。



PART 04 · 第四部分

企业级基础设施与通信协议

把用哪个模型变成可配置项

■ Azure AI Foundry / Azure OpenAI 与私有化模型（如本地 Ollama）异构整合。

■ 推荐端点：gpt-5.5 / phi-4 / Claude，名称通过环境变量可配。

■ 把“用哪个模型”变成配置项，而非硬编码。

承前启后

异构可配的端点抽象，为模块四“按需路由”打下铺垫。

收益 配置驱动的模式选择：切换与灰度都不必改代码。

MCP 让编排器把子任务交给廉价 worker

- MCP 是主流 Agentic IDE 与远程工具服务器对话的标准协议。
- 让昂贵的编排器（大脑）把子任务交给便宜的 worker，且不产生锁定。
- Workshop 中每个角色用 FastMCP over Streamable HTTP 暴露工具集。

为什么重要

标准化 = 可替换、可组合、不锁定，是多智能体的“通用接口”。

价值 用标准协议解耦“大脑”与“工具”，成本与灵活度兼得。

MCP 成为可加固的标准控制平面

MCP 客户端
如 Teams 机器人



Agent 工厂
Azure Container Apps · OpenClaw 沙箱

加固三件套 认证 (Auth) + 严格 allowlist + 最小权限
任何 MCP 客户端都能与 Agent 工厂交互，安全边界由架构保证。

强类型、语义化、最小暴露三原则

强类型 Schema

入参出参有契约，便于校验与审计。

语义插件

把能力封装成可复用、可组合的语义函数。

最小暴露

只给 Agent 暴露必要工具（如 Brain 仅 5 个），越权面最小化。

原则 契约化 + 语义化 + 最小暴露：让工具调用可控、可审计。

前置环境：Python、Docker 与 Foundry

类别	要求
语言	Python 3.12+ （或 .NET 8.0+）
工具	VS Code / Cursor · Git · Docker
账号	GitHub Copilot 订阅 + Copilot CLI 登录 · Microsoft Foundry 项目端点
模型部署	DeepSeek-V4-Flash 与 gpt-5.5 （名称可用环境变量配置）

小结 · 下一站

三条带走，下一站进入 Harness 治理

① 自主闭环

Agentic = ReAct 的思考-行动-观测自主闭环。

② 两种拓扑

Supervisor 与 P2P 各有取舍，按治理与动态性选择。

③ 可运行底座

Workshop 6 模块把概念落成可运行代码。

预告 下一站 · 模块二：深入 Agent Harness 运行时与五大治理支柱。

谢谢 · 模块一结束

进入实操前，先想清楚“大脑之外的骨架”。

下一站 · 模块二 —— Agent Harness 运行时与治理中枢